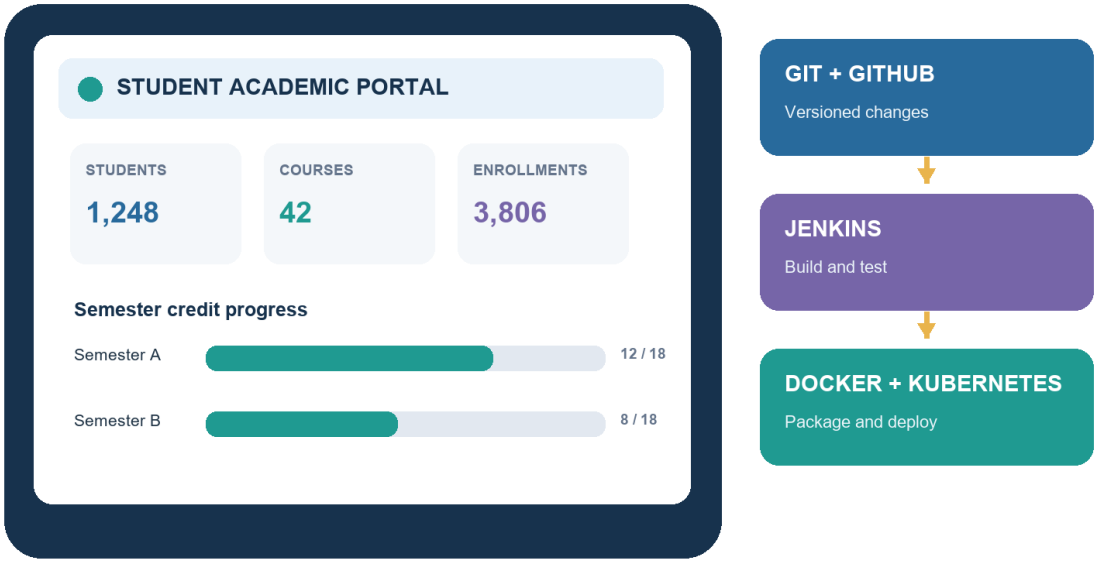
Student Academic Management Platform

DevOps Mini Project Report

Git • Flask • Pytest • Jenkins • Docker • Kubernetes



A student information system delivered through an automated DevOps workflow

Project concept: an academic records service delivered through an automated CI/CD workflow.

PROJECT DETAILS	
Course / Unit	Unit 3: DevOps, CI/CD, Containerization and Application Deployment
Project	Enterprise Student Record and Academic Management Platform
Prepared by	[Student name]
Student ID / Class	[ID] | [Class / section]
Instructor	[Instructor name]
Submission date	[Date]

Replace bracketed fields and add real screenshots, repository details and build results after implementation.

Executive Summary

This mini project applies DevOps practices to an academic records platform for students, faculty, courses, semester enrollments and grades. Flask provides the REST API; a service layer enforces academic rules; SQLite stores local demo records. Pytest verifies behavior and endpoints, Jenkins automates checks, Docker packages the service, and Kubernetes on Minikube manages replicas and health.

The key rule is a maximum of 18 enrolled credits per student in a semester. The design also checks duplicate enrollment, capacity and timetable conflicts, and calculates GPA from completed graded courses. Each code change is versioned, tested, packaged and deployed consistently.

Project outcome A reproducible API with Git history, test results, Jenkins output, Docker image/container and healthy Kubernetes replicas.

Report Guide

CTION	CONTENTS
	Project definition and requirements
	Git and version control
	Application architecture and data model
	Automated tests with Pytest
	Jenkins continuous integration
	Docker containerization
	Kubernetes and Minikube
	Delivery plan, evidence and limitations

1. Project Definition and Scope

1.1 Problem and objective

Academic records may be spread across disconnected spreadsheets and tools. This project demonstrates a centralized service to manage student and course information, record enrollments and grades, and review semester credit totals and GPA. It is an educational prototype, not a production student information system.

- Manage student, faculty and course records.
- Enroll students after eligibility, capacity, schedule and credit checks.
- Record final grades and calculate credit-weighted GPA.
- Provide clear API errors and a health endpoint for deployment probes.
- Demonstrate automated validation, packaging and orchestration.

1.2 Users and quality goals

USER	NEED	SUPPORT
Student	Profile, courses, enrollments and grades	Read API and summaries
Faculty	Course roster and grade submission	Course and grade endpoints
Administrator	Manage people and course records	Create/update operations
Developer/operator	Build, test and deploy	Git, pytest, Jenkins, Docker, Kubernetes

QUALITY	SUCCESS CRITERION
Maintainability	Separate routes, services, data access, models and tests.
Repeatability	Clean checkout installs dependencies and runs tests.
Portability	App runs locally and in a container.
Recovery	Kubernetes restores the three-replica target.
Evidence	History, reports, build output, image and pod status are saved.

2. Topic 1 - Git and Collaborative Version Control

Git records project history; GitHub hosts the shared repository and supports pull-request review. Short-lived branches make topic progression visible and isolate incomplete work.

2.1 Repository structure

```
student-academic-platform/
  app/models/       # Student, Course, Enrollment, Grade
  app/routes/       # Flask API endpoints
  app/services/     # Academic rules and calculations
  app/data/         # Database and repository functions
  tests/            # Unit and API tests
  k8s/              # Deployment and Service manifests
  requirements.txt  Dockerfile  .dockerignore
  Jenkinsfile       README.md
```

2.2 Branch progression

BRANCH	SCOPE	EXIT EVIDENCE
main	Stable milestones	Reviewed release-ready changes
feature/foundation	README, dependencies, Flask skeleton	App starts; health route works

BRANCH	SCOPE	EXIT EVIDENCE
feature/core-app	Models, persistence, services and APIs	Academic rules implemented
feature/automated-tests	Fixtures and pytest cases	Test report generated
feature/jenkins-ci	Jenkinsfile and report publication	Build linked to commit
feature/dockerization	Dockerfile and ignore file	Image and health check work
feature/kubernetes-orchestration	Minikube manifests and checks	Three replicas; self-healing shown

2.3 Workflow

1. Create a feature branch for the next topic.
2. Make a focused change, run checks and review the diff.
3. Commit with a clear message and push the branch.
4. Open a pull request, address review feedback and merge.
5. Record commit and merge evidence in the report.

```
git init
git branch -M main
git checkout -b feature/foundation
git add README.md requirements.txt app/
git commit -m "Create Flask project foundation"
git push -u origin feature/foundation
```

Evidence Repository URL, branch list, commit history and pull request screenshots.

3. Topic 2 - Application Design and Implementation

3.1 Layered architecture

The client calls Flask routes. Routes parse requests and format responses; the service layer applies academic rules; data access functions read or write SQLite. Service-level validation is reusable in API calls and tests.

Application architecture

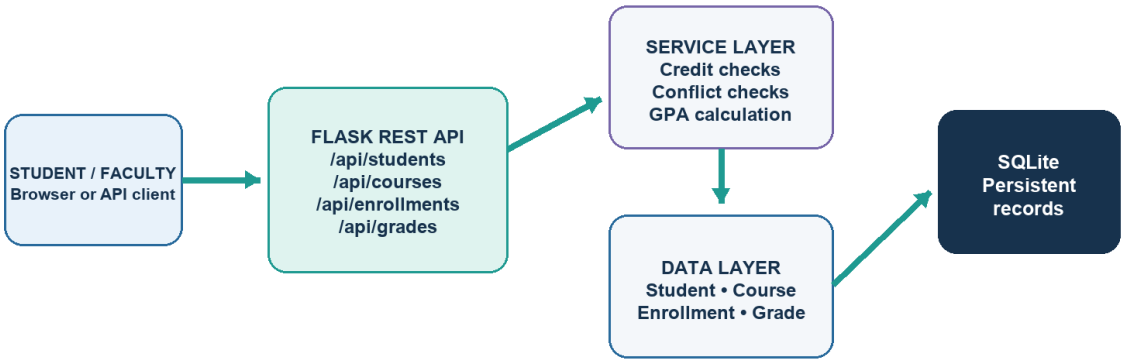


Figure 1. Logical application layers and the path from client to persistent data.

3.2 Entities and relationships

A student has many enrollments, and a course has many enrollments. Enrollment links a student and course for one semester. A grade belongs to a completed enrollment. Foreign keys and uniqueness constraints reinforce these relationships.

Core academic data model

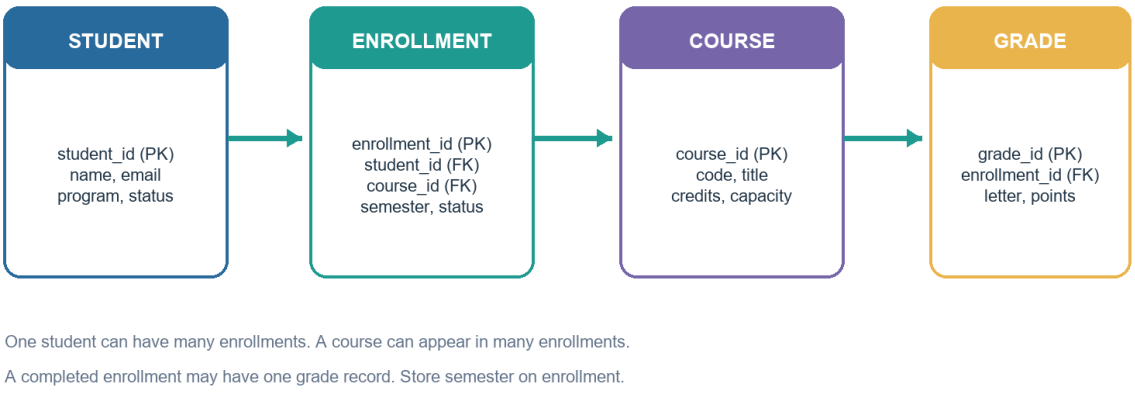


Figure 2. Suggested relational model. PK means primary key; FK means foreign key.

ENTITY	FIELDS	RULE / RELATION

ENTITY	FIELDS	RULE / RELATION
Student	student_id, name, email, program, status	Unique ID/email; many enrollments
Faculty	faculty_id, name, email, department	Assigned to offerings
Course	code, title, credits, capacity, semester	Positive credits; offering identity
Enrollment	student_id, course_id, semester, status	Unique student/course/semester
Grade	enrollment_id, letter, points	One final grade per completed enrollment

3.3 Academic rules

RULE	VALIDATION	RESULT
Credit ceiling	Current credits + requested course <= 18.	Reject when total exceeds 18.
Duplicate	No active same-student/course/semester row.	Conflict; do not duplicate.
Capacity	Active count is below course capacity.	Reject if full.
Schedule	No overlapping meeting interval.	Explain conflicting course.
GPA	Sum(points x credits) / graded credits.	Ignore ungraded courses; define withdrawals.

Example At 15 credits, a 4-credit course is rejected (19 total). A 3-credit course reaches the 18-credit limit if other checks pass.

3.4 API outline

METHOD / PATH	PURPOSE	RESULT
GET /api/health	Health	200, UP status and version
POST /api/students	Register	201 with created record
GET /api/students/{id}	Read student	200 or 404
GET /api/courses?semester=...	List offerings	200 with course list
POST /api/enrollments	Enroll	201; 400 invalid; 409 rule conflict
POST /api/grades	Record grade	201 after validation

METHOD / PATH	PURPOSE	RESULT
GET /api/students/{id}/gpa	Semester GPA	200 with GPA and credits

3.5 Enrollment flow

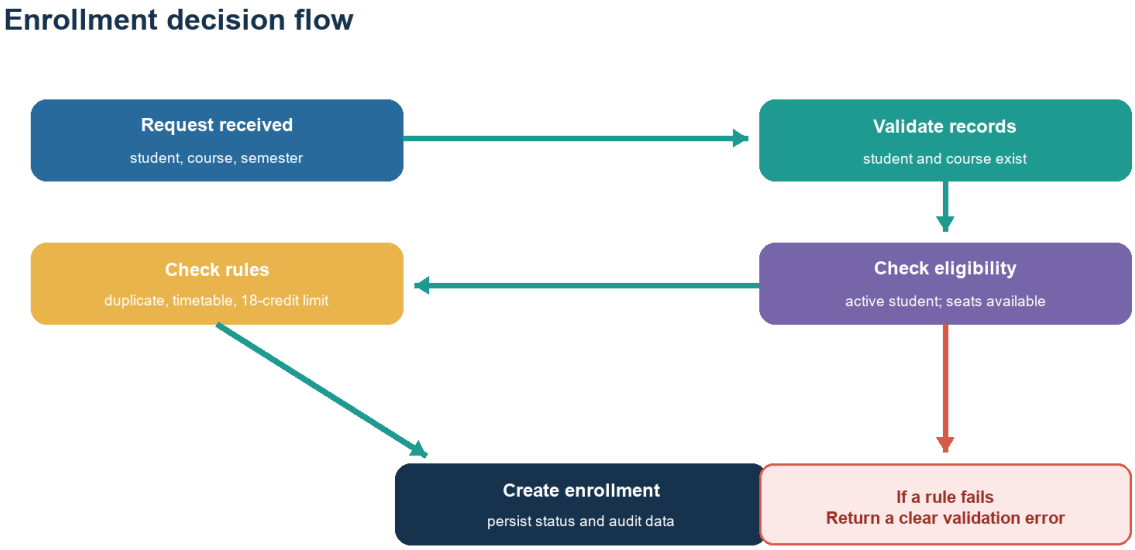


Figure 3. Enrollment is saved only after eligibility and academic checks pass.

4. Topic 3 - Automated Testing with Pytest

Tests preserve academic rules as executable examples. Use service tests for business logic and Flask's test client for API tests. Reset the database before and after each test to prevent cases affecting one another.

LEVEL	SCENARIOS	WHAT IT PROVES
Service	18-credit boundary; overload; duplicate course; weighted GPA.	Rules work in isolation.
Data access	Unique IDs; foreign keys; queries; rollback.	Persistence maintains integrity.
API	Create records; enroll; invalid data; unknown IDs; health.	Routes integrate with data layer.
Container smoke	Start image and request health route.	Packaged service launches.

4.1 Example test

```
def test_credit_limit_rejects_overload(client, student, courses):
    enroll(student.id, courses.three_five_credit_courses)
```

```
    response = client.post("/api/enrollments", json={
        "student_id": student.id,
        "course_id": courses.extra_four_credit_course.id,
        "semester": "2026-Fall",
    })
    assert response.status_code == 409
    assert "18-credit limit" in response.get_json()["error"]
```

4.2 Run and report

```
python -m venv .venv
pip install -r requirements.txt
pytest -v --cov=app --cov-report=term-missing \
  --junitxml=reports/test-results.xml tests/
```

The worked assignment gives 85% as an example coverage threshold. Apply it if required by the course and report measured coverage; do not claim it was met without running tests. Jenkins can publish JUnit XML.

Evidence Passing summary, JUnit report, measured coverage and Jenkins page showing the test stage.

5. Topic 4 - Jenkins Continuous Integration

Jenkins automates checks on pushes or pull requests. Keep the Jenkinsfile in source; keep credentials in Jenkins credential storage, not in the repository.

STAGE	ACTION	FAILURE RESPONSE
Checkout	Fetch exact source revision.	Stop if unavailable.
Environment	Create Python environment; install dependencies.	Stop on setup errors.
Lint	Run Flake8/course-approved checks.	Stop on violations.
Test	Run pytest and export JUnit XML.	Fail build; publish report.
Package	Build image tagged by build/commit.	No package after failed checks.
Deploy	Apply manifests and check rollout.	Report rollout/health failure.

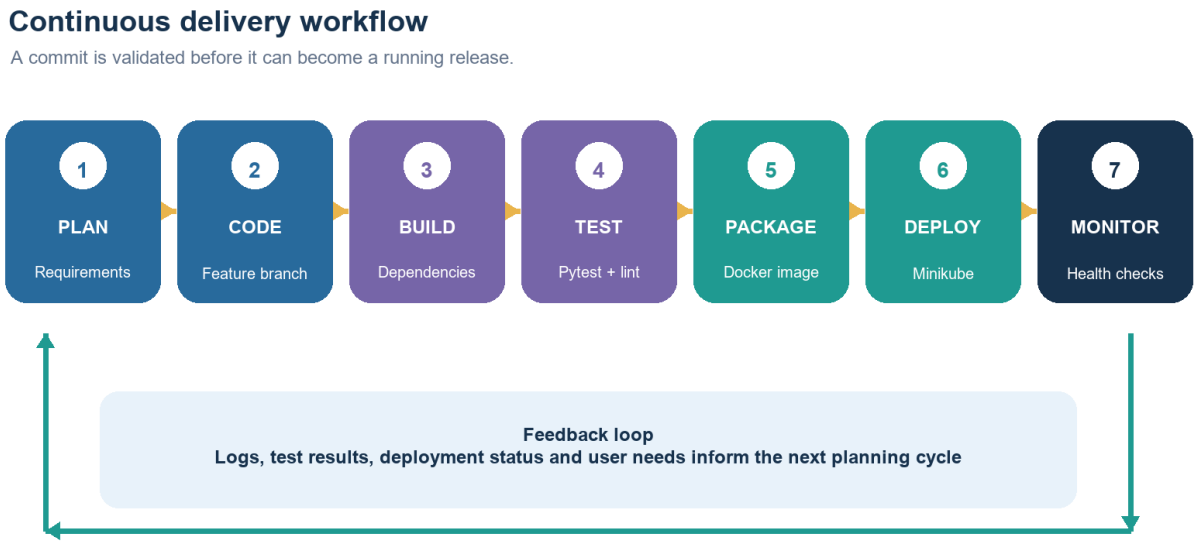


Figure 4. Automated CI stages fit within the DevOps feedback lifecycle.

5.1 Pipeline example

```
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Install') {
      steps { sh 'python3 -m venv .venv && . .venv/bin/activate && pip install -r requirements.txt' }
    }
    stage('Test') {
      steps { sh '. .venv/bin/activate && mkdir -p reports && pytest -v --junitxml=reports/test-results.xml' }
      post { always { junit allowEmptyResults: true, testResults: 'reports/test-results.xml' } }
    }
    stage('Docker build') {
      steps { sh 'docker build -t student-academic-platform:build-tag .' }
    }
  }
}
```

The assignment solution also demonstrates linting, coverage and Kubernetes rollout checks. Adapt Linux shell commands if Jenkins uses a Windows agent. Publish results, use immutable image tags and deploy only after checks pass.

6. Topic 5 - Docker Containerization

Docker packages Python, source code and dependencies in an image. This creates a consistent runtime. Keep container state disposable; persistent student data requires durable storage if extended beyond a demo.

6.1 Dockerfile outline

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PORT=5000
WORKDIR /app
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ app/
COPY run.py .
USER 1000
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:5000/api/health')"
CMD ["python", "run.py"]
```

Copy dependencies before frequently changed source to reuse Docker layers. Use .dockerignore for .git, .venv, __pycache__, local database, test caches and generated reports.

6.2 Build and verify

```
docker build -t student-academic-platform:v1.0.0 .
docker images
docker run -d --name student-academic -p 5000:5000 student-academic-platform:v1.0.0
docker ps
docker logs student-academic
curl http://localhost:5000/api/health
docker stop student-academic
docker rm student-academic
```

- Listen on 0.0.0.0 inside the container; disable debugger.
- Health check should fail when the service is unreachable.
- Use non-root runtime where feasible and omit secrets.
- Capture image, running container and health response.

7. Topic 6 - Kubernetes and Minikube

A Deployment requests three replicas, replaces failed pods and manages rolling updates. A Service provides a stable address. Minikube runs a local cluster for demonstrating the behavior.

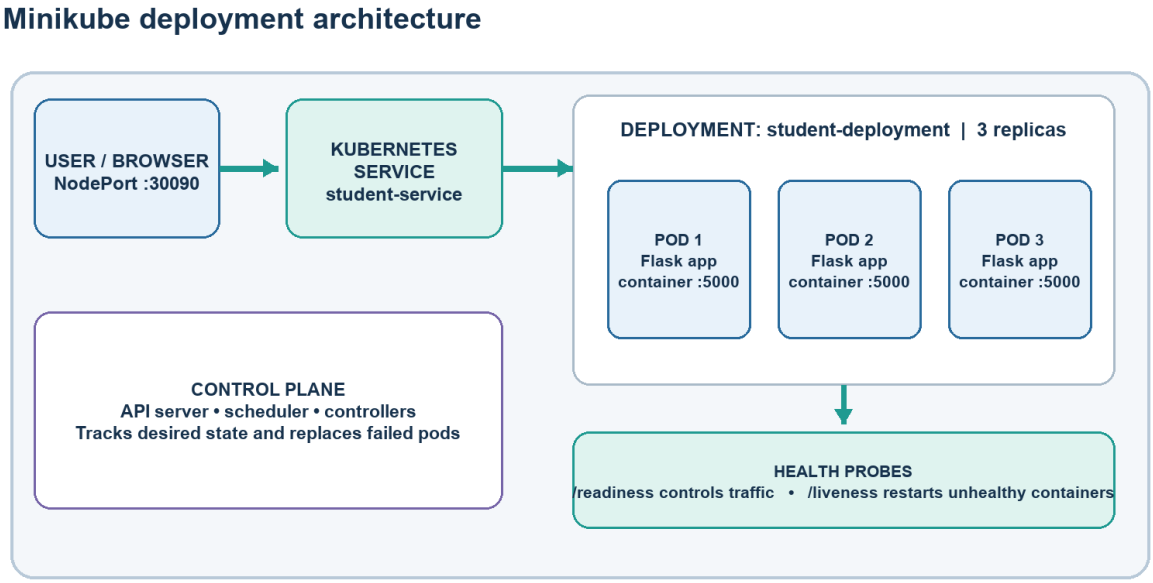


Figure 5. NodePort traffic reaches a stable Service and routes to ready application pods.

7.1 Manifest essentials

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: student-deployment }
spec:
  replicas: 3
  selector: { matchLabels: { app: student-platform } }
  template:
    metadata: { labels: { app: student-platform } }
    spec:
      containers:
      - name: student-app
        image: student-academic-platform:v1.0.0
        imagePullPolicy: IfNotPresent
        ports: [{ containerPort: 5000 }]
        readinessProbe: { httpGet: { path: /api/health, port: 5000 } }
        livenessProbe: { httpGet: { path: /api/health, port: 5000 } }
        resources:
          requests: { cpu: 100m, memory: 128Mi }
          limits: { cpu: 250m, memory: 256Mi }
# service.yaml
apiVersion: v1
kind: Service
metadata: { name: student-service }
spec:
  type: NodePort
  selector: { app: student-platform }
  ports:
  - port: 5000
    targetPort: 5000
    nodePort: 30090
```

7.2 Deploy and verify

```
minikube start --driver=docker
docker build -t student-academic-platform:v1.0.0 .
minikube image load student-academic-platform:v1.0.0
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl rollout status deployment/student-deployment --timeout=120s
kubectl get pods -l app=student-platform -o wide
kubectl get service student-service
minikube service student-service --url
```

Delete one pod for the self-healing demonstration; observe the Deployment replace it and confirm three replicas return. Use kubectl describe and kubectl logs to diagnose pods that are not Ready.

OBJECT	ROLE	PROJECT EXAMPLE
Pod	Runs an application container.	Flask on port 5000.
Deployment	Maintains replicas and rollouts.	Three student-deployment replicas.
Service	Stable endpoint routes to pods.	NodePort 30090.
Readiness	Controls traffic to pods.	Calls /api/health.
Liveness	Restarts unhealthy containers.	Calls health endpoint periodically.

OBJECT	ROLE	PROJECT EXAMPLE
Minikube	Local learning cluster.	Single-machine demonstration.

8. End-to-End Delivery and Submission

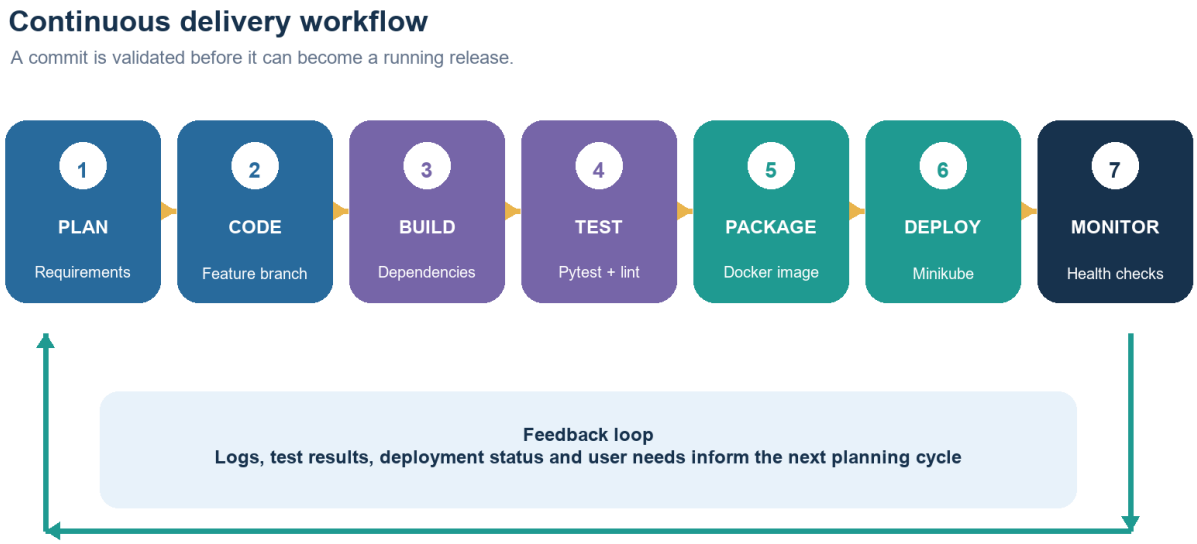


Figure 6. Work moves through planning, Git, automated checks, packaging, deployment and feedback.

6. Write a requirement with a testable acceptance condition.
7. Implement and commit on a feature branch.
8. Jenkins checks out, installs, lints and tests the commit.
9. After checks pass, build a tagged Docker image.
10. Load the image into Minikube and apply Kubernetes state.
11. Check rollout and health; confirm three ready replicas.
12. Save logs, tests, image details and cluster output as evidence.

8.1 Milestones and evidence

MILESTONE	WORK	EXIT CHECK
Foundation	Repo, dependencies, README and health route.	App starts; health succeeds.
Core app	Models, persistence, services and APIs.	Rules and errors shown.
Tests	Fixtures, service and API tests.	Passes; coverage recorded.
Jenkins	Pipeline file and build setup.	Build publishes test report.

MILESTONE	WORK	EXIT CHECK
Docker	Dockerfile, image and smoke test.	Mapped health responds.
Kubernetes	Manifests, Minikube and self-healing.	Three replicas; replacement shown.

8.2 Assignment checklist

- ☐ Repository includes app/, database/data code, tests/, k8s/, Dockerfile, Jenkinsfile, requirements and README.
- ☐ History or pull requests show topic progression.
- ☐ Jenkins screenshot shows successful pipeline and JUnit report.
- ☐ Docker evidence shows tagged image, running container and health result.
- ☐ Minikube evidence shows three healthy pods and replacement of a deleted pod.
- ☐ Report covers setup, architecture, rules, testing, deployment and challenges.
- ☐ Fill cover fields; add actual links, measured results and screenshots.

Academic integrity Run and verify the implementation yourself. Insert your actual results; do not present sample commands as completed work.

9. Limitations and Next Steps

LIMITATION	IMPACT	NEXT STEP
Prototype authentication	Academic records are private.	Add role-based access, login and audit logging.
Ephemeral local data	Pod replacement can lose local records.	Use persistent storage for longer demos.
Basic schedule validation	Conflicts need meeting-time data.	Define intervals and test overlap cases.
Grade policy	GPA depends on institution rules.	Document scale and repeat/withdrawal policy.
Local Minikube	Single node is not production availability.	Use as a learning environment.

Conclusion

A layered Flask service makes academic rules testable, Git provides reviewable history, pytest checks behavior, Jenkins automates quality gates, Docker makes the runtime portable, and Kubernetes maintains desired state. Completion requires working code and evidence that demonstrates each step.

Reference Basis

Prepared from the supplied Assignment 5253 content document and solution document. The assignment uses a library platform as a worked case and asks students to adapt the workflow to a student academic platform. Technical values here are examples; confirm them against the instructor rubric and your implementation.